

EfficientDet 기반 제조 영상 분석 최적화



목차

1. EfficientDet 구조 소개 및 사례
2. 실습 환경 구성 및 데이터셋 준비
3. EfficientDet 사전학습 모델 로드 및 추론
4. EfficientDet 학습 실습
5. 성능 최적화
6. 정리



EfficientDet 기반 제조 영상 분석 최적화

핵심 목표

- EfficientDet 구조와 데이터셋을 이해합니다.
- Custom dataset을 활용해 EfficientDet을 학습하고 추론하는 실습을 진행합니다

EfficientDet 구조 소개 및 사례

EfficientDet 구조 소개 및 사례

5

- EfficientDet : EfficientNet 기반 Object detection 모델, CVPR 2020 에 게재
 - 기존 모델들과 비교해 속도(연산량)과 정확도(mAP)의 균형을 탁월하게 맞춤
 - Compound Scaling을 통해 모델 크기(D0~D7)를 효율적으로 확장 가능
- EfficientDet 모델 scaling

모델	파라미터 수	FPS (Tesla V100)	mAP (COCO)
D0	3.9M	91	33.8
D1	6.6M	74	39.6
D2	8.1M	56	43.0
D3	12M	33	47.5
D4~D7	점차 증가	속도 감소	정확도 ↑

EfficientDet 구조 소개 및 사례

6

- EfficientDet 과 다른 모델 비교
 - 타 모델 대비 적은 연산량 및 모델 크기, 높은 정확도

모델	Params(M)	FLOPs(B)	mAP (COCO)	FPS
YOLOv4	63.9	140.7	43.5	62
RetinaNet	37.7	97.0	39.1	24
EfficientDet-D3	12	8.2	47.5	33

- 제조 현장에서의 사례
 - 적은 모델 크기, 빠른 연산 속도, 높은 정확도로 엠티 디바이스, 제조 현장 등 다양한 분야에서 활용
 - Task, 환경에 따라 scale 선택해서 활용
 - PCB 결함 탐지: 미세한 스크래치, 불량 연결부 탐지에 적합
 - EfficientDet-D0, D1
 - 용접 품질 검사: 용접선 불균형, 결함 부위 실시간 탐지
 - D2 ~ D3 활용

실습 환경 구성 및 데이터셋 준비

실습 환경 구성 및 데이터셋 준비

9

- 관련 라이브러리 설치
 - PyTorch, tool, OpenCV 등 관련 라이브러리 설치
 - 모델은 별도 구현 없이 오픈소스 repository 를 clone해 사용 가능
 - <https://github.com/zylo117/Yet-Another-EfficientDet-Pytorch>

```
>> pip install torch torchvision
>> pip install pycocotools          # coco 데이터셋 관련 라이브러리 설치
>> pip install opencv-python matplotlib tqdm

>> git clone https://github.com/zylo117/Yet-Another-EfficientDet-Pytorch.git
>> cd Yet-Another-EfficientDet-Pytorch
```

실습 환경 구성 및 데이터셋 준비

10

- 관련 라이브러리 설치
 - PyTorch, tool, OpenCV 등 관련 라이브러리 설치
 - 모델은 별도 구현 없이 오픈소스 repository 를 clone해 사용 가능
 - <https://github.com/zylo117/Yet-Another-EfficientDet-Pytorch>

항목	내용
레포지토리 목적	TensorFlow 기반 EfficientDet을 PyTorch 로 재구현한 고성능 object-detection 모델
주요 기능	사전학습 모델 제공, Custom Dataset 학습, 실시간 추론, COCO/VOC 지원
특징	빠르고 효율적인 코드 구조, EfficientDet-D0 ~ D7 전 모델 지원, anchor 설정 가능
라이선스	Apache License 2.0, 개인/상업적 사용 자유, 코드 수정 및 재배포 가능 (라이선스 사본 포함 필요)

- 데이터셋 구조
 - COCO 포맷

```
dataset/  
├─ images/  
│   ├─ train/  
│   └─ val/  
├─ annotations/  
│   ├─ instances_train.json  
│   └─ instances_val.json
```

- 주요 데이터셋 소개
 - YOLO 포맷 외에 COCO 포맷, VOC 포맷 등 다양한 데이터셋의 형태 존재

포맷	주요 사용처	구조 특징	좌표 형식	파일 형식
COCO	학습/평가 표준 (EfficientDet 등 대부분 사용)	하나의 JSON에 전체 어노테이션	x, y, width, height	instances_*.json
VOC	고전 모델들 (Faster R-CNN, SSD 등)	XML 파일로 이미지마다 어노테이션 존재	xmin, ymin, xmax, ymax	*.xml
YOLO	경량 모델들 (YOLOv3 ~ v8)	텍스트 파일로 이미지마다 별도 저장	class x_center y_center width height (정규화됨)	*.txt
VisDrone	드론 영상 객체 탐지	CSV 형식 유사한 텍스트	x, y, width, height 외에 여러 필드	*.txt
Open Images	구글에서 제공한 대형 데이터셋	CSV 기반	xmin, xmax, ymin, ymax	train-annotations-bbox.csv 등

- COCO 포맷
 - 가장 널리 사용되는 포맷 (EfficientDet, Detectron2, MMDetection 등 주요 라이브러리에서 기본 지원)
 - 하나의 json 파일에 모든 이미지/라벨 포함
 - 각 object 들은 category_id, bounding box, segmentation 정보를 포함
 - json 파일로 한 번에 관리가 쉽고, segmentation, keypoint 등 task 확장에 용이해 많이 사용

```
{  
  "images": [ {"id": 1, "file_name": "0001.jpg", ...} ],  
  "annotations": [  
    {"image_id": 1, "bbox": [x, y, w, h], "category_id": 3, ...}  
  ],  
  "categories": [ {"id": 3, "name": "car"}, ... ]  
}
```

- VOC 포맷
 - Pascal Visual Object Classes
 - 이미지 한 장마다 라벨 역할을 하는 xml 파일이 존재
 - xml 파일 내에 object 태그로 라벨링

```
<object>
  <name>car</name>
  <bndbox>
    <xmin>50</xmin> <ymin>50</ymin>
    <xmax>200</xmax> <ymax>200</ymax>
  </bndbox>
</object>
```

- YOLO 포맷
 - 이미지 한 장마다 라벨 역할을 하는 txt 파일이 존재
 - 정규화된 중심좌표 기반의 값들이 나열 (클래스 ID, x, y, w, h)

```
2 0.473 0.612 0.186 0.239
```

EfficientDet 사전학습 모델 로드 및 추론 I

EfficientDet 사전학습 모델 로드 및 추론

19

- 오픈소스 EfficientDet repository 활용, pretrained 모델 load
 - Pretrained weight 다운로드

```
>> git clone https://github.com/zylo117/Yet-Another-EfficientDet-Pytorch.git
>> cd Yet-Another-EfficientDet-Pytorch
>> pip install -r requirements.txt

>> mkdir -p weights
>> wget https://github.com/zylo117/Yet-Another-Efficient-
Pytorch/releases/download/1.0/efficientdet-d0.pth -O weights/efficientdet-
d0.pth
```

Pretrained weights and benchmark

The performance is very close to the paper's, it is still SOTA.

The speed/FPS test includes the time of post-processing with no jit/data precision trick.

coefficient	pth_download	GPU Mem(MB)	FPS	Extreme FPS (Batchsize 32)	mAP 0.5:0.95(this repo)	mAP 0.5:0.95(official)
D0	efficientdet-d0.pth	1049	36.20	163.14	33.1	33.8
D1	efficientdet-d1.pth	1159	29.69	63.08	38.8	39.6
D2	efficientdet-d2.pth	1321	26.50	40.99	42.1	43.0
D3	efficientdet-d3.pth	1647	22.73	-	45.6	45.8
D4	efficientdet-	1903	14.75	-	48.8	49.4

EfficientDet 사전학습 모델 로드 및 추론

20

- 모델 초기화 및 로드

```
import torch
from backbone import EfficientDetBackbone

compound_coef = 0
model = EfficientDetBackbone(compound_coef=0, num_classes=90)
```



모델 크기 조절
d0 ~ d7



모델이 학습한 데이터셋의 Class 개수

```
model.load_state_dict(torch.load(f'weights/efficientdet-d0.pth', map_location='cpu'))
model.requires_grad_(False)
model.eval()
```

- 모델 초기화 및 로드

```
import torch
from backbone import EfficientDetBackbone

compound_coef = 0
model = EfficientDetBackbone(compound_coef=0, num_classes=90)
```

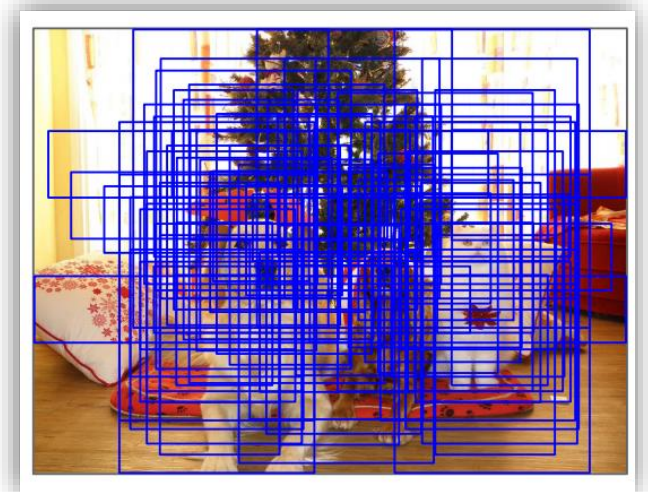
모델 크기 조절
d0 ~ d7

모델이 학습한 데이터셋의 Class 개수

compound_coef	모델 이름	입력 크기	성능	속도
0	EfficientDet-D0	512x512	빠름, 낮은 정확도	가볍고 빠름
1	EfficientDet-D1	640x640	중간	보통
7	EfficientDet-D7	1536x1536	느리지만 고정확도	무거움

```
model.load_state_dict(torch.load(f'weights/efficientdet-d0.pth', map_location='cpu'))
model.requires_grad_(False)
model.eval()
```

- 모델 초기화 및 로드
 - Anchor
 - 모델의 위치를 추정할 때 사용하는 기준이 되는 box
 - 이 anchor 를 기준으로 크기와 위치를 예측해서 실제 bounding box를 예측
 - Anchor ratios : Anchor 의 가로 세로 비율을 설정
 - Anchor scale : 여러 크기를 갖는 object 탐지를 위해 크기를 설정



```
anchor_ratios = [(1.0, 1.0), (1.4, 0.7), (0.7, 1.4)]  
anchor_scales = [2 ** 0, 2 ** (1.0 / 3.0), 2 ** (2.0 / 3.0)]
```

- 모델 초기화 및 로드
 - Task 및 데이터에 맞게 threshold, iou_threshold 설정
 - 연산 관련 파라미터 세팅

```
threshold = 0.2          # 모델이 예측한 object 중, 신뢰도 0.2 이상인 결과만 출력
iou_threshold = 0.2      # 중복된 박스 처리를 위한 파라미터, 겹치는 박스 중 IoU가 0.2 이상인 경우,
                        # 중복 탐지로 간주하고 하나만 남김 (낮으면 진짜 객체 missing / 높으면 중복)
use_cuda = True          # CUDA 사용 (GPU 연산 여부) 결정
Use_float16 = False      # 16 bit 연산 사용 여부
cudnn.fastest = True     # cuDNN 라이브러리에서 가장 빠른 알고리즘 선택
```

- 입력 shape 확인 및 전처리
 - 모델의 학습 사이즈 (d0 : (512 x 512), d1 : (640 x 640), ...) 에 맞추어 입력 tensor shape 조정
 - 최종 shape 은 (batch_size, channel, height_size, width_size) 가 되어야 함
 - 예) d0 모델에 rgb 이미지 1장을 테스트 하는 경우 → torch.tensor([1, 3, 512, 512])
 - preprocess 함수를 활용해 전처리

```
from utils.utils import preprocess

img_path = 'test/img.png'
input_sizes = [512, 640, 768, 896, 1024, 1280, 1280, 1536, 1536]
input_size = input_sizes[compound_coef] if force_input_size is None else force_input_size
ori_imgs, framed_imgs, framed metas = preprocess(img_path, max_size=input_size)

if use_cuda:
    x = torch.stack([torch.from_numpy(fi).cuda() for fi in framed_imgs], 0)
else:
    x = torch.stack([torch.from_numpy(fi) for fi in framed_imgs], 0)

x = x.to(torch.float32 if not use_float16 else torch.float16).permute(0, 3, 1, 2)
```

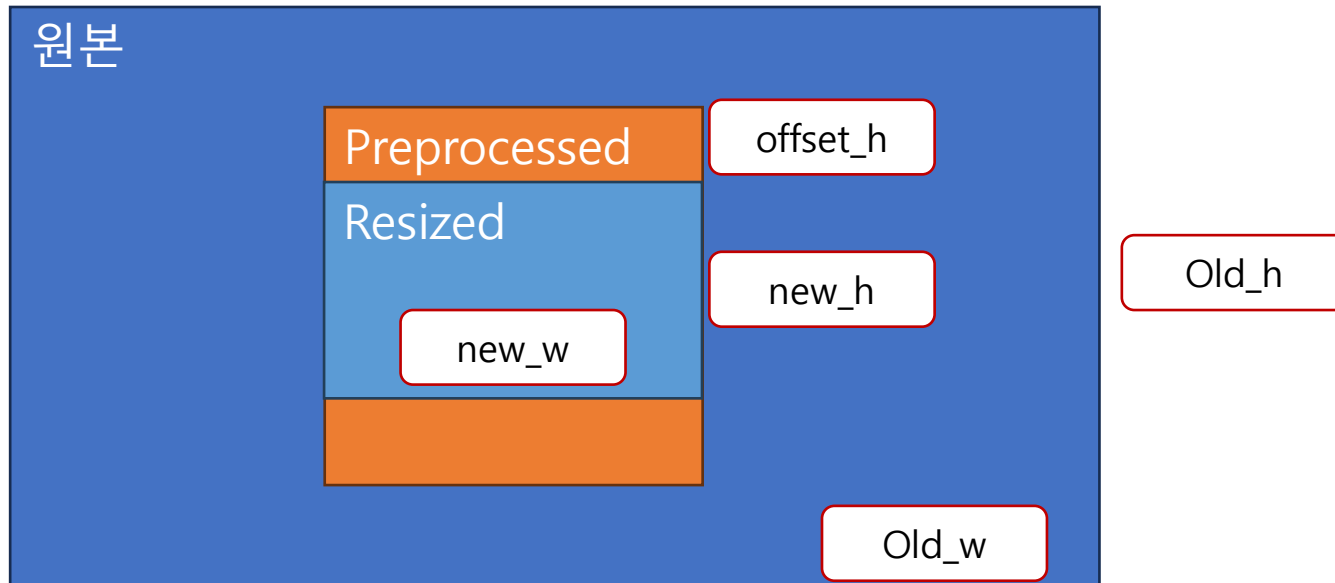
- Framed_metas
 - 전처리 과정에서 이미지 변환을 기록하는 메타데이터

```
from utils.utils import preprocess
```

```
>> framed_metas
```

```
>> [(512, 288, 1920, 1080, 0, 224)]
```

```
(new_w, new_h, old_w, old_h, offset_w, offset_h)
```



- 전처리한 입력 tensor 를 model 에 넣어 inference
 - features, regression, classification, anchors 4개의 값이 return
 - regression : Anchor box를 얼마나 이동/확대할지를 예측한 값
 - classification : Anchor box 마다 클래스들의 확률
 - anchors : 기준이 되는 anchor box 들

```
if use_cuda:
    model = model.cuda()
if use_float16:
    model = model.half()

with torch.no_grad():
    features, regression, classification, anchors = model(x)
```

- Features
 - 중간 단계에서 생성된 feature map 저장
- Regression
 - Anchor box 마다 좌표를 얼마나 조정해야 할지 (dx, dy, dw, dh) return
- Classification
 - Anchor box 마다 90 개의 object 별로 detect probability return
- Anchors
 - 49104 개의 anchor box 마다 4개의 좌표 (x1, y1, x2, y2) 값 return
 - 여러 개의 feature map에서 anchor 생성 : $(64*64 + 32*32 + 16*16 + 8*8 + 4*4) * 9 = 49104$

```
with torch.no_grad():  
    features, regression, classification, anchors = model(x)
```

```
>> features[0].shape : torch.Size([1, 64, 64, 64])  
>> features[4].shape : torch.Size([1, 64, 4, 4])  
>> regression[0].shape : torch.Size([49104, 4])  
>> classification.shape : torch.Size([1, 49104, 90])  
>> anchors.shape : torch.Size([1, 49104, 4])
```

- 전처리한 입력 tensor 를 model 에 넣어 inference
 - features, regression, classification, anchors 4개의 값이 return
 - Return 된 값을 우리가 알아볼 수 있는 bounding box의 좌표, confidence, class 정보로 변환
 - postprocess 함수를 활용해 간단히 적용

```
from utils.utils import postprocess

with torch.no_grad():
    features, regression, classification, anchors = model(x)

    regressBoxes = BBoxTransform() # 모델이 예측한 regression 값을 실제 bounding box 좌표로 변환
    clipBoxes = ClipBoxes()        # 모델이 예측한 object의 box가 이미지 범위를 벗어나지 않도록 조정

    out = postprocess(x,
                      anchors, regression, classification,
                      regressBoxes, clipBoxes,
                      threshold, iou_threshold)

>> print(out)
>> [{'rois': array([[ 69.27405 , 54.400414, 438.81613 , 292.56516 ]], dtype=float32),
      'class_ids': array([5]), 'scores': array([0.9569864], dtype=float32)}]
```

- 전처리한 입력 tensor 를 model 에 넣어 inference
 - 시각화

```
def display(preds, imgs, imshow=True, imwrite=False):
    for i in range(len(imgs)):
        if len(preds[i]['rois']) == 0:
            continue

        imgs[i] = imgs[i].copy()

        for j in range(len(preds[i]['rois'])):
            x1, y1, x2, y2 = preds[i]['rois'][j].astype(np.int64)
            obj = obj_list[preds[i]['class_ids'][j]]
            score = float(preds[i]['scores'][j])
            plot_one_box(imgs[i], [x1, y1, x2, y2],
                          label=obj, score=score, color=color_list[get_index_label(obj, obj_list)])

        if imshow:
            cv2.imshow('img', imgs[i])
            cv2.waitKey(0)

        if imwrite:
            cv2.imwrite(f'test/img_inferred_d{compound_coef}_this_repo_{i}.jpg', imgs[i])
```

EfficientDet 사전학습 모델 로드 및 추론

30

- 전처리한 입력 tensor 를 model 에 넣어 inference
 - 시각화



EfficientDet 사전학습 모델 로드 및 추론

31

- 영상에서 test
 - 영상은 이미지의 연속이기 때문에 이미지와 모든 process 동일
 - 영상 관련 코드 추가 시 동일하게 진행 가능



```
obj_list = ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat', 'traffic light',
            'fire hydrant', '', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep',
            'cow', 'elephant', 'bear', 'zebra', 'giraffe', '', 'backpack', 'umbrella', '', '', 'handbag', 'tie',
            'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove',
            'skateboard', 'surfboard', 'tennis racket', 'bottle', '', 'wine glass', 'cup', 'fork', 'knife', 'spoon',
            'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut',
            'cake', 'chair', 'couch', 'potted plant', 'bed', '', 'dining table', '', '', 'toilet', '', 'tv',
            'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink',
            'refrigerator', '', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier',
            'toothbrush']

# tf bilinear interpolation is different from any other's, just make do
input_sizes = [512, 640, 768, 896, 1024, 1280, 1280, 1536, 1536]
input_size = input_sizes[compound_coef] if force_input_size is None else force_input_size

# load model
model = EfficientDetBackbone(compound_coef=compound_coef, num_classes=len(obj_list))
model.load_state_dict(torch.load(f'weights/efficientdet-d{compound_coef}.pth'))
model.requires_grad_(False)
model.eval()

# Box
regressBoxes = BBoxTransform()
clipBoxes = ClipBoxes()
```

```
# Video capture
cap = cv2.VideoCapture(video_src)

while True:
    ret, frame = cap.read()
    if not ret:
        break
    # frame preprocessing
    ori_imgs, framed_imgs, framed metas = preprocess_video(frame, max_size=input_size)

    x = x.to(torch.float32 if not use_float16 else torch.float16).permute(0, 3, 1, 2)
    # model predict
    with torch.no_grad():
        features, regression, classification, anchors = model(x)

        out = postprocess(x,
                           anchors, regression, classification,
                           regressBoxes, clipBoxes,
                           threshold, iou_threshold)

    # result
    out = invert_affine(framed metas, out)
    img_show = display(out, ori_imgs)

    # show frame by frame
    cv2.imshow('frame',img_show)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```



EfficientDet utils 함수 알아보기

- preprocess

- 이미지 파일 경로를 입력받아 모델 입력에 맞게 정규화 및 리사이즈

```
def preprocess(*image_path, max_size=512, mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)):
    ori_imgs = [cv2.imread(img_path) for img_path in image_path]
    normalized_imgs = [(img[..., ::-1] / 255 - mean) / std for img in ori_imgs]
    imgs_meta = [aspectaware_resize_padding(img, max_size, max_size,
                                             means=None) for img in normalized_imgs]

    framed_imgs = [img_meta[0] for img_meta in imgs_meta]
    framed metas = [img_meta[1:] for img_meta in imgs_meta]

    return ori_imgs, framed_imgs, framed metas
```

- preprocess_video

- 영상 프레임을 직접 받아 정규화 및 리사이즈

```
def preprocess_video(*frame_from_video, max_size=512, mean=(0.406, 0.456, 0.485), std=(0.225,
0.224, 0.229)):
    ori_imgs = frame_from_video
    normalized_imgs = [(img[..., ::-1] / 255 - mean) / std for img in ori_imgs]
```

- aspectaware_resize_padding
 - 주어진 이미지의 aspect ratio 를 유지하면서 width x height 크기로 리사이즈
 - 나머지는 padding (mean)으로 채워주는 함수 → 모델 입력 시 정규화 되기 때문

```
def preprocess(*image_path, max_size=512, mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)):
    imgs_meta = [aspectaware_resize_padding(img, max_size, max_size,
                                             means=None) for img in normalized_imgs]
```

```
def aspectaware_resize_padding(image, width, height, interpolation=None, means=None):
    ...

    canvas = np.zeros((height, height, c), np.float32)
    if means is not None:
        canvas[...] = means

    image = cv2.resize(image, (new_w, new_h))

    padding_h = height - new_h
    padding_w = width - new_w

    canvas[:new_h, :new_w] = image
    return canvas, new_w, new_h, old_w, old_h, padding_w, padding_h,
```

- invert_affine
 - 모델의 output bounding box 를 원래 이미지 해상도로 복원
 - meta 에 담긴 resize 정보 (new_w, old_w, padding) 를 이용해 좌표 보정

```
def invert_affine(metas: Union[float, list, tuple], preds):  
    for i in range(len(preds)):  
        if len(preds[i]['rois']) == 0:  
            continue  
        else:  
            if metas is float:  
                preds[i]['rois'][:, [0, 2]] = preds[i]['rois'][:, [0, 2]] / metas  
                preds[i]['rois'][:, [1, 3]] = preds[i]['rois'][:, [1, 3]] / metas  
            else:  
                new_w, new_h, old_w, old_h, padding_w, padding_h = metas[i]  
                preds[i]['rois'][:, [0, 2]] = preds[i]['rois'][:, [0, 2]] / (new_w / old_w)  
                preds[i]['rois'][:, [1, 3]] = preds[i]['rois'][:, [1, 3]] / (new_h / old_h)  
    return preds
```

```
[{'rois': array([[333.9105, 239.9248, 424.3782, 287.47534],  
                [314.58044, 129.695, 351.32635, 163.35178],  
                [350.18134, 165.45305, 433.36963, 241.53828],  
                [130.67781, 232.14659, 207.67549, 287.5896],  
                [362.00366, 135.14462, 408.66394, 169.6925],  
                [ 56.523525, 167.49182, 129.53528, 224.73615],  
                [414.87433, 173.18341, 506.79755, 242.55426],  
                [ 0.8860817, 233.43503, 48.69839, 277.26898],  
                [ 80.50641, 246.80035, 122.748184, 286.69604],  
                [232.99245, 165.09128, 285.26987, 247.47049])}]
```

framed_metas

[(512, 288, 1920, 1080, 0, 224)]

EfficientDet utils 함수 알아보기

- BBoxTransform
 - Regression 값을 bounding box 로 디코딩
 - EfficientDet 이 출력한 예측값인 dy, dx, dh, dw 를 anchor box 를 기준으로 bounding box 좌표 복원

```
y_centers_a = (anchors[..., 0] + anchors[..., 2]) / 2 # anchor 중심 y
x_centers_a = (anchors[..., 1] + anchors[..., 3]) / 2 # anchor 중심 x
ha = anchors[..., 2] - anchors[..., 0] # anchor 높이
wa = anchors[..., 3] - anchors[..., 1] # anchor 너비

h = regression[..., 2].exp() * ha # 예측된 box 높이
w = regression[..., 3].exp() * wa # 예측된 box 너비

y_centers = regression[..., 0] * ha + y_centers_a
x_centers = regression[..., 1] * wa + x_centers_a

# box 좌표 (xmin, ymin, xmax, ymax) 계산
return torch.stack([xmin, ymin, xmax, ymax], dim=2)
```

EfficientDet utils 함수 알아보기

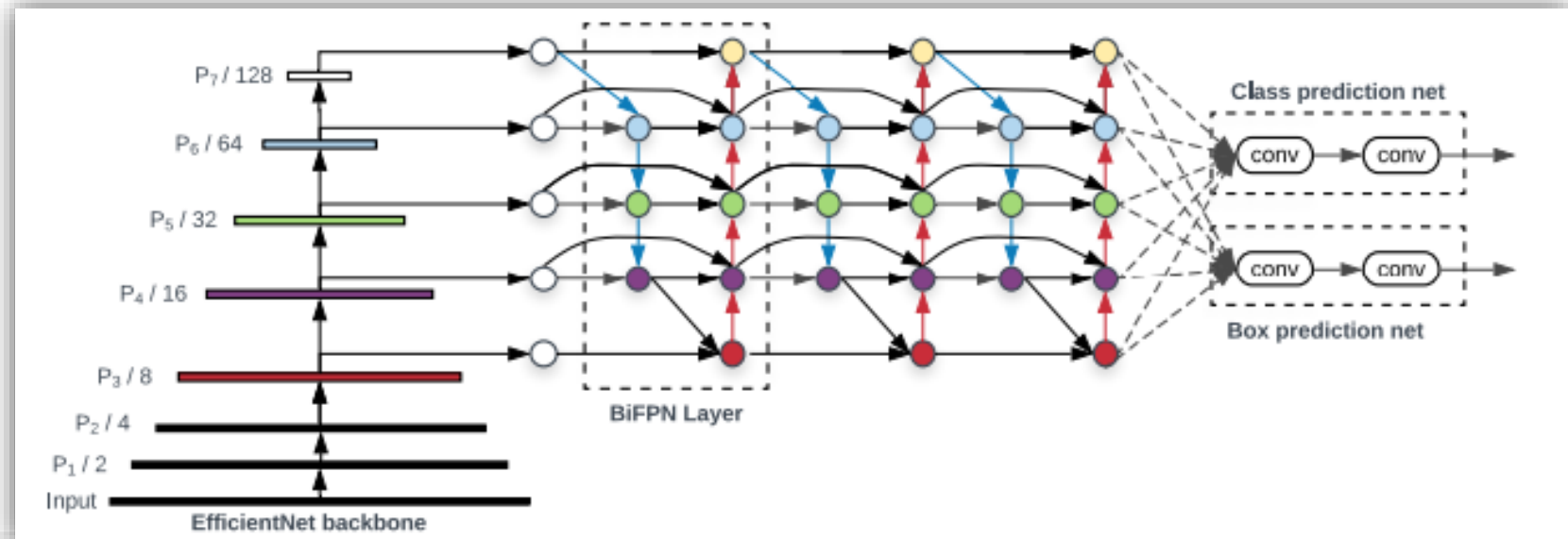
- ClipBoxes
 - Bounding box 를 이미지 크기에 맞춰 clamping
 - EfficientDet 이 출력한 예측값인 dy, dx, dh, dw 를 anchor box 를 기준으로 bounding box 좌표 복원

```
boxes[:, :, 0] = torch.clamp(boxes[:, :, 0], min=0)          # xmin >= 0
boxes[:, :, 1] = torch.clamp(boxes[:, :, 1], min=0)          # ymin >= 0
boxes[:, :, 2] = torch.clamp(boxes[:, :, 2], max=width - 1)  # xmax <= width
boxes[:, :, 3] = torch.clamp(boxes[:, :, 3], max=height - 1) # ymax <= height
```

EfficientDet utils 함수 알아보기

- Anchors
 - 다양한 크기, 비율의 anchor box 생성
 - P3 ~ P7, 다양한 크기의 feature map 마다 다양한 scale/ratio 조합의 anchor box 생성

```
self.pyramid_levels = [3, 4, 5, 6, 7]      # FPN 레벨  
self.strides = [8, 16, 32, 64, 128]        # 각 레벨의 stride  
self.scales = [1.0, 2 ** (1/3), 2 ** (2/3)] # anchor box 크기  
self.ratios = [(1.0, 1.0), (1.4, 0.7), (0.7, 1.4)] # 종횡비
```



EfficientDet utils 함수 알아보기

- 전처리 process

- 입력 이미지 → Anchors → EfficientDet head → BBoxTransform → ClipBoxes

↓
다양한 위치·크기·비율의
anchor box 생성
(anchor proposal)

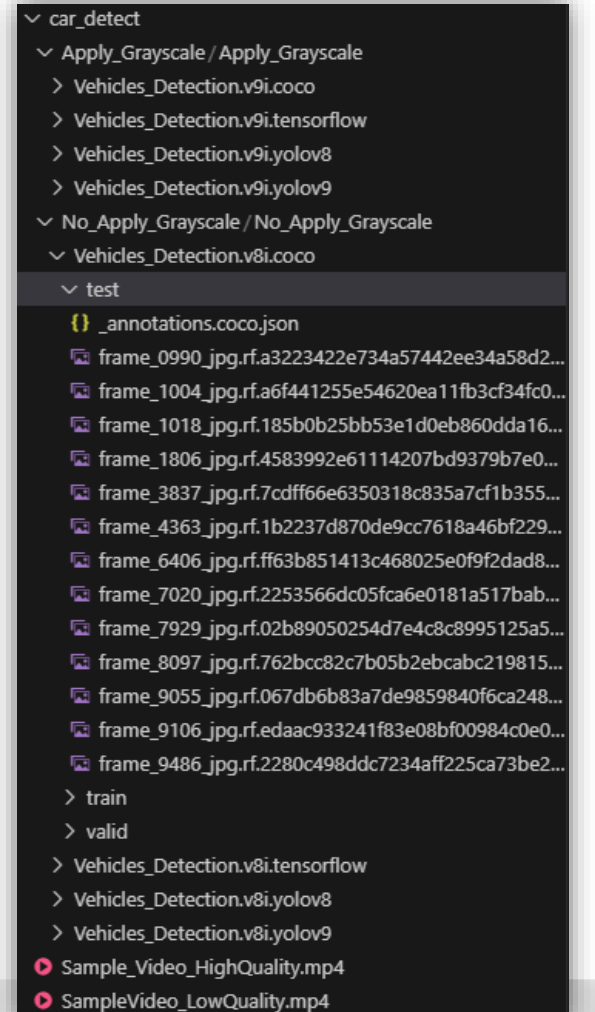
↓
각 anchor에 대해 (class,
bbox regression) 예측

↓
regression 값을 실제
box 좌표로 변환

↓
box가 이미지 밖으로 나
가지 않도록 클램핑

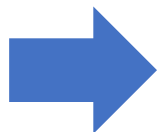
EfficientDet 학습 실습

- 데이터셋 구성
 - <https://www.kaggle.com/datasets/pkdarabi/vehicle-detection-image-dataset>
 - Train / Val / Test 셋으로 구성
 - Set 마다 라벨링된 annotation 파일 존재 (Coco format 지원)



- 데이터셋 구성
 - 학습을 위해 변경 필요

```
▼ car_detect
  ▼ Apply_Grayscale / Apply_Grayscale
    > Vehicles_Detection.v9i.coco
    > Vehicles_Detection.v9i.tensorflow
    > Vehicles_Detection.v9i.yolov8
    > Vehicles_Detection.v9i.yolov9
  ▼ No_Apply_Grayscale / No_Apply_Grayscale
    ▼ Vehicles_Detection.v8i.coco
      ▼ test
        {} _annotations.coco.json
        frame_0990_jpg.rf.a3223422e734a57442ee34a58d2...
        frame_1004_jpg.rf.a6f441255e54620ea11fb3cf34fc...
        frame_1018_jpg.rf.185b0b25bb53e1d0eb860dda16...
        frame_1806_jpg.rf.4583992e61114207bd9379b7e0...
        frame_3837_jpg.rf.7cdf66e6350318c835a7cf1b355...
        frame_4363_jpg.rf.1b2237d870de9cc7618a46bf229...
        frame_6406_jpg.rf.ff63b851413c468025e0f9f2dad8...
        frame_7020_jpg.rf.2253566dc05fca6e0181a517bab...
        frame_7929_jpg.rf.02b89050254d7e4c8c8995125a5...
        frame_8097_jpg.rf.762bcc82c7b05b2ebcabc219815...
        frame_9055_jpg.rf.067db6b83a7de9859840f6ca248...
        frame_9106_jpg.rf.edaac933241f83e08bf00984c0e0...
        frame_9486_jpg.rf.2280c498ddc7234aff225ca73be2...
      > train
      > valid
      > Vehicles_Detection.v8i.tensorflow
      > Vehicles_Detection.v8i.yolov8
      > Vehicles_Detection.v8i.yolov9
      ▶ Sample_Video_HighQuality.mp4
      ▶ SampleVideo_LowQuality.mp4
```



```
▼ my_car_detect_proj
  ▼ annotations
    {} instances_test.json U
    {} instances_train.json U
    {} instances_valid.json U
  ▼ test
    frame_0990_jpg.rf.a3223422e734a57442ee34a... U
    frame_1004_jpg.rf.a6f441255e54620ea11fb3cf... U
    frame_1018_jpg.rf.185b0b25bb53e1d0eb860d... U
    frame_1806_jpg.rf.4583992e61114207bd9379... U
    frame_3837_jpg.rf.7cdf66e6350318c835a7cf1... U
    frame_4363_jpg.rf.1b2237d870de9cc7618a46... U
    frame_6406_jpg.rf.ff63b851413c468025e0f9f2... U
    frame_7020_jpg.rf.2253566dc05fca6e0181a51... U
    frame_7929_jpg.rf.02b89050254d7e4c8c89951... U
    frame_8097_jpg.rf.762bcc82c7b05b2ebcabc21... U
    frame_9055_jpg.rf.067db6b83a7de9859840f6c... U
    frame_9106_jpg.rf.edaac933241f83e08bf0098... U
    frame_9486_jpg.rf.2280c498ddc7234aff225ca7... U
  > train
  > valid
```

모델 평가 및 예측 시각화

- FPS, mAP 등의 metric을 활용해 속도 및 정확도 평가

Average Precision	(AP) @[IoU=0.50:0.95 area= all maxDets=100]	= 0.555
Average Precision	(AP) @[IoU=0.50 area= all maxDets=100]	= 0.754
Average Precision	(AP) @[IoU=0.75 area= all maxDets=100]	= 0.644
Average Precision	(AP) @[IoU=0.50:0.95 area= small maxDets=100]	= 0.564
Average Precision	(AP) @[IoU=0.50:0.95 area=medium maxDets=100]	= 0.484
Average Precision	(AP) @[IoU=0.50:0.95 area= large maxDets=100]	= 0.549
Average Recall	(AR) @[IoU=0.50:0.95 area= all maxDets= 1]	= 0.575
Average Recall	(AR) @[IoU=0.50:0.95 area= all maxDets= 10]	= 0.640
Average Recall	(AR) @[IoU=0.50:0.95 area= all maxDets=100]	= 0.640
Average Recall	(AR) @[IoU=0.50:0.95 area= small maxDets=100]	= 0.564
Average Recall	(AR) @[IoU=0.50:0.95 area=medium maxDets=100]	= 0.573
Average Recall	(AR) @[IoU=0.50:0.95 area= large maxDets=100]	= 0.638